

Reviewer Pack — Lovász-Theta of Term-Conflict Graph Bounds Monotone k-CLIQUE...

Entry 6227dd6720d4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 16:26:48 UTC

Lovász-Theta of Term-Conflict Graph Bounds Monotone k-CLIQUE DNF

Entry ID: 6227dd6720d4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-18 16:26:48 UTC

1. Conjecture

Field A (mathematical branch): Lovász theta / degree-2 Sum-of-Squares SDP relaxation, accessed through the Hoffman ratio bound $\alpha_H(G) := -\lambda_{\min} \cdot |V| / (\lambda_{\max} - \lambda_{\min})$ on independence number (Lovász 1979 ‘On the Shannon capacity of a graph’; Hoffman 1970; Schrijver 1979; Goemans-Williamson 1995). This is the canonical degree-2 SOS object: $\vartheta(G) = \max \sum x_i$ s.t. positive-semidefinite X with $X_{ii} = x_i$ and $X_{ij} = 0$ on edges. Applied STRUCTURALLY to the term-conflict graph G_F of a monotone DNF F (a polymatroid-expansion object on the term family), the invariant is a property of the DNF wiring not of its truth table — two monotone DNFs with the same Boolean function can have wildly different G_F , shielding from Razborov-Rudich natural proofs. The functional uses real eigenvalue extraction and ratio (no F_q polynomial extension exists, since the spectral ratio $\lambda_{\min} / (\lambda_{\max} - \lambda_{\min})$ is not preserved under polynomial ring lifts), Aaronson-Wigderson algebrization-safe. Distinct from blacklisted Bonami-Beckner stable-rank / term-pair noise-ratio (those use a hypercontractive Gram kernel, not an SDP on the conflict graph), Cauchy-mean-width of the minterm hull (convex-geometry intrinsic 1-volume), Forman-Ricci curvature (Bochner edge-vertex sum), and F_2 -corank of minterm incidence (binary-matroid representability). An arXiv search ‘Lovász theta’ AND (‘monotone DNF’ OR ‘k-CLIQUE indicator’ OR ‘term-conflict graph’) returns 0 direct hits and <5 adjacent papers (Lee-Schreiber-Theis theta-rank, never on monotone DNF term families). **Field B** (complexity object): Monotone DNF leaf size $L(F) = \sum_i |T_i|$ for the k-CLIQUE indicator F^*_v on K_v with $k = \lceil \log_2 v \rceil$ and $n = v(v-1)/2$ edge variables (Razborov 1985 / Andreev / Karchmer-Wigderson regime). The $k = \log v$ scaling makes the canonical minterm DNF of size $(v \text{ choose } k)$ super-polynomial in n , the standard MONOTONE CLIQUE focus’s stepping stone toward the Razborov $n^{\Omega(k)}\{1/4\}$ monotone lower bound and a $\tilde{V}^0$ separation via Buss-Pudlák-Krajíček-style approximator counting.

Statement:

For a monotone DNF $F = \bigvee_{i=1}^v T_i$ on n variables, define the term-conflict graph G_F on vertex set $[s]$ with edges $E(G_F) = \{\{i,j\} : T_i \cap T_j \neq \emptyset\}$, let A_F be its $\{0,1\}$ -adjacency matrix with extreme eigenvalues $\lambda_1 \geq \lambda_s$, and set $\alpha_H(F) := -\lambda_s \cdot s / (\lambda_1 - \lambda_s)$ if $\lambda_1 > 0$ else $\alpha_H(F) := s$. Define the measure $\mu(F) := s / (\alpha_H(F) \cdot (1 + \log_2(1 + \max_i |T_i|)))$. Conjecture: (i) for every two monotone DNFs F, G on disjoint term-lists, $\mu(F \vee G) \leq \mu(F) + \mu(G)$ and $\mu(F \wedge G) \leq \mu(F) + \mu(G)$ (where $F \wedge G$ ’s term list is $\{T_i \cup T'_j\}_{i,j}$); (ii) for every monotone DNF F with at most $s \leq \text{poly}(n)$ terms and max term size $\leq C \cdot \log n$, $\mu(F) \leq C' \cdot \log_2 n$ with absolute $C' > 0$; (iii) for the k-CLIQUE indicator F^*_v on K_v with $k = \lceil \log_2 v \rceil$, expressed as the canonical disjunction of k-clique minterms in

edge-variables, $\mu(F^*_v) \geq c \cdot v$ for an absolute constant $c > 0$. A single random-poly-size DNF with $\mu > 8 \cdot \log_2 n$, or a k-CLIQUE canonical DNF F^*_v with $\mu(F^*_v) < v/16$, refutes.

Rationale (proposer’s reasoning):

Lovász theta and its Hoffman spectral surrogate are the canonical degree-2 SOS lift of independence number; applied to the term-conflict graph of a monotone DNF they probe the polymatroid expansion of the term family — exactly the quantity that Razborov-style approximators try to capture but on the SYNTACTIC side. The k-CLIQUE canonical DNF has terms = k-cliques in K_v , inducing a dense Johnson/Kneser-type conflict graph whose spectrum is highly concentrated ($\lambda_1 \approx k \cdot (v-k) \cdot C$, $\lambda_s \approx -\Theta(k)$), so $\alpha_H \approx s/(v-k)$ and μ scales linearly in v . Generic poly-size monotone DNFs with short terms have low-density conflict graphs with $\alpha_H \approx s/(\max\text{-degree}+1)$, keeping μ at log-scale; the bridge therefore exploits the SOS-2 spectral fingerprint of polymatroidal term overlap without ever touching the truth table.

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 38184f59704aa41d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds: (a) for every $v \in \{5, \dots, 10\}$, min-seed $\mu(F^*_v)/v \geq 1/16$; (b) for every random monotone DNF instance (all n, k, s , seed combos), $\mu(F)/\log_2 n \leq 8$; (c) Spearman $\rho(\mu(F^*_v), v) \geq 0.7$. All three must hold for support.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Lovász theta monotone DNF term conflict graph - Hoffman bound monotone circuit lower bound clique - spectral SDP relaxation monotone formula leaf size k-clique

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
```

```

import math
import itertools
from collections import defaultdict

def matrix_multiply(A, B):
    n = len(A)
    m = len(B[0])
    p = len(B)
    result = [[0.0 for _ in range(m)] for _ in range(n)]
    for i in range(n):
        for j in range(m):
            for k in range(p):
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_norm(M):
    if not M:
        return 0.0
    return math.sqrt(sum(sum(x**2 for x in row) for row in M))

def power_iteration(M, max_iter=200, tol=1e-6):
    n = len(M)
    if n == 0:
        return 0.0, [0.0] * n
    b_k = [random.random() for _ in range(n)]
    b_k_norm = matrix_norm(b_k)
    if b_k_norm == 0:
        return 0.0, [0.0] * n
    b_k = [x / b_k_norm for x in b_k]

    for _ in range(max_iter):
        b_k1 = [sum(M[i][j] * b_k[j] for j in range(n)) for i in range(n)]
        b_k1_norm = matrix_norm(b_k1)
        if b_k1_norm == 0:
            return 0.0, [0.0] * n
        b_k1 = [x / b_k1_norm for x in b_k1]

        if abs(b_k1_norm - b_k_norm) < tol:
            break
        b_k = b_k1
        b_k_norm = b_k1_norm

    return b_k_norm, b_k

def compute_extreme_eigenvalues(M):
    if not M:
        return 0.0, 0.0
    lambda_max, _ = power_iteration(M)
    n = len(M)
    I = [[1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]
    M_shifted = [[M[i][j] - lambda_max * I[i][j] for j in range(n)] for i in range(n)]
    lambda_min, _ = power_iteration(M_shifted)
    lambda_min += lambda_max
    return lambda_max, lambda_min

def generate_k_clique_dnf(v, k):

```

```

edges = list(itertools.combinations(range(v), 2))
terms = list(itertools.combinations(edges, k))
return terms

def build_term_conflict_graph(terms):
    s = len(terms)
    adj_matrix = [[0.0 for _ in range(s)] for _ in range(s)]
    for i in range(s):
        for j in range(i + 1, s):
            if any(e in terms[j] for e in terms[i]):
                adj_matrix[i][j] = 1.0
                adj_matrix[j][i] = 1.0
    return adj_matrix

def compute_mu(terms, adj_matrix):
    s = len(terms)
    if s == 0:
        return 0.0
    lambda_max, lambda_min = compute_extreme_eigenvalues(adj_matrix)
    if lambda_max == 0:
        alpha_h = s
    else:
        alpha_h = -lambda_min * s / (lambda_max - lambda_min)
    max_term_size = max(len(term) for term in terms) if terms else 0
    if max_term_size == 0:
        return 0.0
    mu = s / (alpha_h * (1 + math.log2(1 + max_term_size)))
    return mu

def generate_random_dnf(n, s, k):
    terms = []
    for _ in range(s):
        term = random.sample(range(n), k)
        terms.append(term)
    return terms

def run_trial(seed):
    random.seed(seed)
    metric_values = []
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    # Test k-CLIQUE canonical DNF
    for v in range(5, 11):
        k = math.ceil(math.log2(v))
        terms = generate_k_clique_dnf(v, k)
        adj_matrix = build_term_conflict_graph(terms)
        mu = compute_mu(terms, adj_matrix)
        metric_values.append(mu / v)
        instances_tested += 1
        if mu < v / 16:
            conjecture_holds = False
            counterexample = f"k-CLIQUE canonical DNF with v={v}, mu={mu} < v/16"

    # Test random monotone DNFs

```

```

for n in [15, 20, 25, 30, 40]:
    for s in [n, 2*n, 4*n, 8*n]:
        if s > 256:
            continue
        k = math.ceil(math.log2(n))
        terms = generate_random_dnf(n, s, k)
        adj_matrix = build_term_conflict_graph(terms)
        mu = compute_mu(terms, adj_matrix)
        metric_values.append(mu / math.log2(n))
        instances_tested += 1
        if mu > 8 * math.log2(n):
            conjecture_holds = False
            counterexample = f"Random monotone DNF with n={n}, s={s}, mu={mu} > 8*log2(n)"

if not metric_values:
    return {
        "metric_name": "mu_ratio",
        "metric_value": 0.0,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "No instances tested"
    }

avg_metric = sum(metric_values) / len(metric_values)
return {
    "metric_name": "mu_ratio",
    "metric_value": avg_metric,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = sys.argv[1:] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41]
    trials = []
    for seed in seeds:
        trial = run_trial(seed)
        trials.append(trial)
        print(f"TRIAL: {trial}")

    metric_values = [trial["metric_value"] for trial in trials if trial["metric_value"] is not None]
    if not metric_values:
        print("RESULT: INCONCLUSIVE reason=no_metric_values")
        sys.exit(0)

    mean_metric = sum(metric_values) / len(metric_values)
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(1 for trial in trials if trial["conjecture_holds"]) / len(trials)

    if any(not trial["conjecture_holds"] for trial in trials):
        first_failing_seed = next(seed for seed, trial in zip(seeds, trials) if not trial["conjecture_holds"])
        counterexample = next(trial["counterexample"] for trial in trials if not trial["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    else:

```

```
print("RESULT: INCONCLUSIVE reason=insufficient_support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6bb3cd60.py", line 160, in <module>
    trial = run_trial(seed)
            ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6bb3cd60.py", line 116, in run_trial
    mu = compute_mu(terms, adj_matrix)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6bb3cd60.py", line 86, in compute_mu
    lambda_max, lambda_min = compute_extreme_eigenvalues(adj_matrix)
                             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6bb3cd60.py", line 59, in compute_extreme_eigenvalues
    lambda_max, _ = power_iteration(M)
                   ^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6bb3cd60.py", line 37, in power_iteration
    b_k_norm = matrix_norm(b_k)
               ^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6bb3cd60.py", line 30, in matrix_norm
    return math.sqrt(sum(sum(x**2 for x in row) for row in M))
                       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6bb3cd60.py", line 30, in <genexpr>
    return math.sqrt(sum(sum(x**2 for x in row) for row in M))
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

TypeError: 'float' object is not iterable

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in the `power_iteration` routine (`matrix_norm` received a 1D vector instead of a 2D matrix), producing no data on either the random-DNF or k-CLIQUE criteria. No pre-registered condition can be evaluated. | next: Fix `matrix_norm` to handle 1D vectors (or use `numpy.linalg.eigvalsh` for the symmetric {0,1}-adjacency matrix) and rerun the 30-seed protocol.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	214374
2	preregistration	claude_max	opus	0	0	7752
3	novelty	claude_max	opus	0	0	4641
4	novelty	claude_max	opus	0	0	7419
5	test_gen	mistral	codestral-latest	0	0	325094
6	test_gen	mistral	codestral-latest	0	0	313969
7	test_gen	mistral	codestral-latest	0	0	317329
8	test_gen	mistral	codestral-latest	0	0	318454
9	judge	claude_max	opus	0	0	5649

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1514680 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/6227dd6720d4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6227dd6720d4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/6227dd6720d4.tar.gz` (if generated)